

Módulo 4 — Decisões (Condicionais)

Jornada DEV START · Aula 4 · 16/07 (quinta) · Programa START — TOTVS Paulista
Material entregue ao final da Aula 4. Guarde para consultar sempre que precisar.



Onde você está na jornada

```
1 → 2 → 3 → [4] → 5 → 6 | 7 → 8 → 9 → 10
Fundamentos (Harbour) | Protheus (ADVPL)
▲ você está aqui
```

Você já sabe guardar dados em variáveis e combiná-los com operadores. Hoje o programa **ganha inteligência**: ele passa a **tomar decisões** e seguir caminhos diferentes conforme a situação. É aqui que o código deixa de ser uma receita fixa e começa a “pensar”. 🧠



Roteiro da aula (2h)

Bloco	Tempo	O quê
Retomada	~10 min	Revisão do Módulo 3 (variáveis, operadores) + correção de exercício-chave
Conteúdo	~60 min	IF/ELSEIF/ELSE , IIF() , IF aninhado e DO CASE com exemplos ao vivo
Mão na massa	~40 min	Construir juntos um menu de lanchonete com DO CASE
Q&A / networking	~10 min	Dúvidas abertas e integração da turma

🎯 Objetivos do módulo

Ao final desta aula você será capaz de: - Usar `IF/ELSEIF/ELSE/ENDIF` para decisões simples, compostas e encadeadas - Escrever decisões compactas com `IIF()` - Usar `DO CASE` para seleção múltipla - Entender **quando usar cada estrutura** - Aplicar decisões em problemas reais (aprovação, faixas de valor, menus)

🔄 Retomando a aula anterior

No **Módulo 3** você aprendeu a **guardar** dados (variáveis com prefixos: `nSalario`, `cNome`, `lAtivo`, `dData`) e a **combiná-los** com operadores. Dois grupos de operadores são a base de hoje:

Operadores de comparação	Operadores lógicos
<code>></code> <code><</code> <code>>=</code> <code><=</code> <code>==</code> (igual) <code>!=</code> (diferente)	<code>.AND.</code> (e) · <code>.OR.</code> (ou) · <code>.NOT.</code> (não)

Toda comparação devolve um valor **lógico**: `.T.` (verdadeiro) ou `.F.` (falso).

```
? (7 >= 5)           // .T.  
? (10 == 3)          // .F.  
? (nIdade >= 18 .AND. lCNH) // .T. só se as DUAS forem verdadeiras
```

Guarde esta ideia: **uma decisão é apenas uma pergunta cuja resposta é `.T.` ou `.F.`** — e é sobre isso que este módulo se apoia.

4.1 Por que precisamos de decisões?

Até agora seus programas executam sempre as mesmas instruções, do início ao fim — como uma estrada reta. Mas problemas reais têm **bifurcações**: o caminho muda conforme a situação.

🗺️ **Analogia:** pense num GPS. Se a rua está livre, ele segue reto; **se** há um engarrafamento, ele **recalcula** e desvia. O programa faz o mesmo: olha a condição e escolhe por onde ir.

Exemplos do mundo real: - Se o saldo for suficiente, autorize o pagamento; senão, recuse. - Se a nota for maior ou igual a 7, aprove; senão, reprove. - Se o produto escolhido for 1, mostre

R\$ 10; se for 2, R\$ 25; se for 3, R\$ 18.

Para isso usamos as **estruturas de decisão** (também chamadas de seleção ou desvio condicional).

 Fluxograma de uma decisão IF: uma condição que separa dois caminhos

4.2 IF — Decisão Simples

A estrutura mais básica: executa um bloco **apenas se** a condição for verdadeira.

Sintaxe

```
IF <condição>
    // bloco executado se a condição = .T.
ENDIF
```

Exemplo

```
FUNCTION Main()
    LOCAL nA, nB, nResultado

    ACCEPT "Valor A: " TO nA
    ACCEPT "Valor B: " TO nB
    nA := Val(nA)
    nB := Val(nB)

    nResultado := nA + nB

    IF nResultado > 10
        QQOut("Resultado " + Str(nResultado) + " é maior que 10!")
    ENDIF

    QQOut("Programa encerrado.")

    RETURN NIL
```

Fluxo:

```
    [condição = .T.?]
      ↓ Sim      ↓ Não
[executa bloco]  [pula bloco]
      ↓          ↓
    [continua o programa]
```

Repare: a linha `Q0ut("Programa encerrado.")` roda **sempre**, com ou sem o `IF`. O bloco condicional é só o que está **entre** `IF` e `ENDIF`.

4.3 IF/ELSE — Decisão Composta

Executa um bloco se a condição for verdadeira e **outro bloco** se for falsa. Dois caminhos, sempre um deles acontece.



Analogia: “Se chover, levo guarda-chuva; **senão**, levo óculos de sol.” Nunca os dois, nunca nenhum — exatamente um.

Sintaxe

```
IF <condição>
    // bloco executado se .T.
ELSE
    // bloco executado se .F.
ENDIF
```

Exemplo: aprovação escolar

```
FUNCTION Main()
    LOCAL nMedia

    ACCEPT "Média final: " TO nMedia
    nMedia := Val(nMedia)

    IF nMedia >= 7
        Q0ut("Situação: APROVADO")
        Q0ut("Parabéns! Média: " + Str(nMedia, 5, 1))
    ELSE
        Q0ut("Situação: REPROVADO")
        Q0ut("Recuperação necessária. Média: " + Str(nMedia, 5, 1))
    ENDIF

    RETURN NIL
```

Forma compacta: IIF()

Quando a decisão serve só para **escolher um valor**, use a função `IIF()` (Inline IF). Ela cabe em uma linha e recebe três argumentos: `IIF(condição, valor_se_.T., valor_se_.F.)`.

```
LOCAL cSituacao := IIF(nMedia >= 7, "Aprovado", "Reprovado")
LOCAL nDesconto := IIF(nIdade >= 60, nValor * 0.125, 0)
```

- 💡 Use `IIF()` para **atribuições curtas**. Se cada caminho precisa de várias linhas, prefira o `IF/ELSE` tradicional — fica mais legível.

4.4 IF/ELSEIF/ELSE — Decisão Encadeada

Quando é preciso testar **várias condições em sequência**, encadeamos com `ELSEIF`.

Sintaxe

```
IF <condição 1>
    // bloco 1
ELSEIF <condição 2>
    // bloco 2
ELSEIF <condição 3>
    // bloco 3
ELSE
    // bloco padrão (nenhuma condição foi verdadeira)
ENDIF
```

⚠ **Importante:** assim que uma condição é verdadeira, o bloco dela executa e as demais **não são testadas**. Por isso a **ordem importa** — coloque as faixas na sequência certa.

Exemplo: reajuste de salário

```
FUNCTION Main()
  LOCAL nSalario, nNovoSalario, nReajuste

  ACCEPT "Salário atual: R$ " TO nSalario
  nSalario := Val(nSalario)

  IF nSalario < 500
    nReajuste := 0.15    // 15%
  ELSEIF nSalario <= 1000
    nReajuste := 0.10    // 10%
  ELSE
    nReajuste := 0.05    // 5%
  ENDIF

  nNovoSalario := nSalario * (1 + nReajuste)

  QOut("Salário atual: R$ " + Str(nSalario, 8, 2))
  QOut("Reajuste:      " + Str(nReajuste * 100, 5, 1) + "%")
  QOut("Novo salário:  R$ " + Str(nNovoSalario, 8, 2))

  RETURN NIL
```

Exemplo: classificação por faixa etária

```
FUNCTION Main()
  LOCAL nIdade, cFaixa

  ACCEPT "Idade: " TO nIdade
  nIdade := Val(nIdade)

  IF nIdade < 0
    cFaixa := "Idade inválida"
  ELSEIF nIdade <= 12
    cFaixa := "Criança"
  ELSEIF nIdade <= 17
    cFaixa := "Adolescente"
  ELSEIF nIdade <= 59
    cFaixa := "Adulto"
  ELSE
    cFaixa := "Idoso"
  ENDIF

  QOut("Faixa etária: " + cFaixa)

  RETURN NIL
```

Note como cada **ELSEIF** já assume que o anterior falhou: quando o código chega em **nIdade <= 17**, ele **já sabe** que a idade é maior que 12. Não precisamos escrever **nIdade >= 13 .AND. nIdade <= 17**.

4.5 IF aninhado (Nested IF)

Às vezes é preciso testar uma condição **dentro** de outra. Chamamos isso de IF aninhado.

```
FUNCTION Main()
    LOCAL nSalario, cTemFilho, lTemFilho, nBeneficio

    ACCEPT "Salário: R$ " T0 nSalario
    ACCEPT "Tem filho? (S/N): " T0 cTemFilho
    nSalario := Val(nSalario)
    lTemFilho := (Upper(cTemFilho) == "S")

    IF nSalario <= 1500
        IF lTemFilho
            nBeneficio := 300    // salário baixo + filho = maior benefício
        ELSE
            nBeneficio := 150    // salário baixo, sem filho
        ENDIF
    ELSE
        IF lTemFilho
            nBeneficio := 100    // salário alto + filho
        ELSE
            nBeneficio := 0      // salário alto, sem filho
        ENDIF
    ENDIF

    QOut("Benefício: R$ " + Str(nBeneficio, 6, 2))

    RETURN NIL
```

⚠ **Cuidado com a legibilidade:** quando o aninhamento passa de 2–3 níveis, o código fica difícil de ler. Nesses casos, considere usar **DO CASE** ou extrair a lógica para uma função (assunto do Módulo 6).

4.6 DO CASE — Seleção Múltipla

Ideal quando você compara **uma mesma variável** com **vários valores possíveis**.



Analogia: é o painel de um elevador. Você aperta **um** botão (o andar) e ele leva exatamente ao destino daquele botão. Cada **CASE** é um botão; o **OTHERWISE** é o “andar padrão” para quando ninguém aperta nada válido.

Sintaxe

```
DO CASE
  CASE <condição 1>
    // bloco 1
  CASE <condição 2>
    // bloco 2
  CASE <condição 3>
    // bloco 3
  OTHERWISE
    // bloco padrão
ENDCASE
```

OTHERWISE é opcional — equivale ao **ELSE** do **IF** .

Exemplo: calculadora com menu

```
FUNCTION Main()
  LOCAL nA, nB, cOp

  ACCEPT "Valor A: " TO nA
  ACCEPT "Valor B: " TO nB
  ACCEPT "Operação (+, -, *, /): " TO cOp
  nA := Val(nA)
  nB := Val(nB)

  DO CASE
    CASE cOp == "+"
      QOut("Resultado: " + Str(nA + nB, 10, 2))
    CASE cOp == "-"
      QOut("Resultado: " + Str(nA - nB, 10, 2))
    CASE cOp == "*"
      QOut("Resultado: " + Str(nA * nB, 10, 2))
    CASE cOp == "/"
      IF nB == 0
        QOut("Erro: divisão por zero!")
      ELSE
        QOut("Resultado: " + Str(nA / nB, 10, 2))
      ENDIF
    OTHERWISE
      QOut("Operação inválida: " + cOp)
  ENDCASE

  RETURN NIL
```

Exemplo: menu do sistema

```
FUNCTION Main()
  LOCAL cOpcao

  QOut("=== SISTEMA DE CADASTRO ===")
  QOut("1 - Incluir")
  QOut("2 - Consultar")
  QOut("3 - Alterar")
  QOut("4 - Excluir")
  QOut("0 - Sair")
  QOut("")
  ACCEPT "Escolha uma opção: " TO cOpcao

  DO CASE
    CASE cOpcao == "1"
      QOut("Abrindo tela de inclusão...")
    CASE cOpcao == "2"
      QOut("Abrindo consulta...")
    CASE cOpcao == "3"
      QOut("Abrindo alteração...")
    CASE cOpcao == "4"
      QOut("Abrindo exclusão...")
    CASE cOpcao == "0"
      QOut("Encerrando o sistema. Até logo!")
    OTHERWISE
      QOut("Opção inválida! Digite 0, 1, 2, 3 ou 4.")
  ENDCASE

  RETURN NIL
```



Esse padrão de **menu com DO CASE** é exatamente o que você vai reencontrar dentro do Protheus para navegar entre rotinas. Aprender bem agora rende muito lá na frente.



Menu de opções tratado com DO CASE, cada opção levando a uma ação

4.7 IF vs DO CASE: quando usar cada um?

Situação	Use
Uma condição simples	IF/ENDIF
Dois caminhos possíveis (sim/não)	IF/ELSE/ENDIF
Faixas de valores (ex.: salário < 500, <= 1000, > 1000)	IF/ELSEIF/ELSE/ENDIF
Comparar uma variável com vários valores exatos	DO CASE
Menu com opções discretas	DO CASE

Regra prática: se você compara a **mesma variável** com muitos valores, prefira **DO CASE**. Se as condições são **expressões diferentes**, use **IF/ELSEIF**.

4.8 Exemplo integrado: nota e conceito

Este exemplo reúne tudo: validação com **IF**, classificação com **DO CASE** e um **IIF()**.

```

FUNCTION Main()
    LOCAL nNota, cConceito, cSituacao

    ACCEPT "Nota (0 a 10): " TO nNota
    nNota := Val(nNota)

    IF nNota < 0 .OR. nNota > 10
        QOut("Nota inválida!")
        RETURN NIL
    ENDIF

    DO CASE
        CASE nNota >= 9
            cConceito := "A"
        CASE nNota >= 7
            cConceito := "B"
        CASE nNota >= 5
            cConceito := "C"
        CASE nNota >= 3
            cConceito := "D"
        OTHERWISE
            cConceito := "E"
    ENDCASE

    cSituacao := IIF(nNota >= 5, "Aprovado", "Reprovado")

    QOut("Nota: " + Str(nNota, 5, 1))
    QOut("Conceito: " + cConceito)
    QOut("Situação: " + cSituacao)

    RETURN NIL

```

Repare no **RETURN NIL** dentro do **IF** : se a nota é inválida, o programa **encerra na hora** e nem chega no **DO CASE** . Essa técnica de “sair cedo” (early return) mantém o código limpo.

👉 Mão na massa (em aula)

Vamos construir juntos, passo a passo, um **menu de lanchonete**:

Crie um arquivo `lanchonete.prg` que: 1. Exiba um cardápio com 3 opções e a opção de sair: `=== LANCHONETE DEV === 1 - Hambúrguer .... R$ 18,00 2 - Batata frita ... R$ 12,00 3 - Refrigerante ... R$ 7,00 0 - Sair` 2. Leia a opção do cliente com `ACCEPT`. 3. Use `DO CASE` para mostrar o preço do item escolhido (ou “Até logo!” no `0`). 4. No `OTHERWISE`, avise que a opção é inválida. 5. **Incremento (se der tempo):** peça a **quantidade** e, com um `IF`, dê **10% de desconto** se o total passar de R\$ 50,00.

Faremos linha a linha — e no fim cada um roda o seu cardápio. 🍔

📌 Referência rápida do Módulo 4

```
// Decisão simples
IF <condição>
    // ...
ENDIF

// Decisão composta
IF <condição>
    // ...
ELSE
    // ...
ENDIF

// Decisão encadeada (a 1ª verdadeira vence; ordem importa)
IF <cond 1>
    // ...
ELSEIF <cond 2>
    // ...
ELSE
    // ...
ENDIF

// Decisão compacta (para escolher um valor)
cResultado := IIF(<condição>, <valor se .T.>, <valor se .F.>)

// Seleção múltipla (mesma variável, vários valores)
DO CASE
CASE <cond 1>
    // ...
CASE <cond 2>
    // ...
OTHERWISE
    // ...
ENDCASE
```

Operadores usados nas condições:

```
> < >= <= == != // comparação
.AND. .OR. .NOT. // lógicos
```



Exercícios

Os exercícios desta aula estão em `exercicios.md` (nesta mesma pasta). **Prazo sugerido:** até a **Aula 5 (17/07)**.



Para a próxima aula

Módulo 5 — Repetição (loops). Você aprendeu a fazer o programa **escolher** um caminho. Agora vai aprender a fazê-lo **repetir** tarefas: somar 1000 números, validar entradas até virem corretas e percorrer listas — sem reescrever código. Entram em cena o `WHILE`, o `FOR/NEXT` ... e o temido **loop infinito** (e como evitá-lo).



Antes da Aula 5: rode o seu `lanchonete.prg`. Travou em algo? Poste no Discord `#duvidas-parte1-harbour` — a gente destrava junto.